

Karol de Grote

Data & A.I. 6

LLM Sentiment Project 2025

Made by:
Liza Kolosova
Khaled Asfour

Introduction

Problem description

The growing amount of user reviews makes manual analysis difficult and time-consuming, creating a need for automated methods to extract meaningful insights efficiently.

Aspect-Based Sentiment Analysis (ABSA) addresses this need by identifying specific aspects or features mentioned in text and determining the sentiment expressed toward each. This approach is useful in domains like customer reviews, product feedback, social media analysis, and tourism, where you need to constantly check people's opinions about your product.

Project goals

Implement and evaluate three distinct Aspect-Based Sentiment Analysis (ABSA) approaches: a Transformer-based model (TransformerABSA), a Lexicon-based method (LexiconABSA), and a Large Language Model-based solution (LLMABSA).

Compare these methods by examining their strengths and weaknesses, performance characteristics, and overall accuracy on relevant datasets. Additionally, analyze which approach is most suitable for various real-world scenarios, considering factors such as data availability, domain specificity, computational resources, and the complexity of sentiment expressions.

Methodology

2.1 LexiconABSA

The LexiconABSA implementation uses a rule-based and linguistic approach to Aspect-Based Sentiment Analysis (Spacy). It leverages linguistic dependency parsing, lexical sentiment scores, and hand-crafted heuristics to extract and evaluate aspect terms from text.

First, the model uses SpaCy's transformer-based pipeline (`en_core_web_trf`) to perform syntactic parsing, part-of-speech tagging, and dependency analysis. From these annotations, it identifies candidate aspect terms, which are typically nouns or noun phrases appearing as syntactic heads of noun chunks (e.g., "food", "service", "battery life"). Determiners and pronouns are removed to ensure that extracted candidates represent concrete concepts.

For each aspect candidate, the system searches for sentiment-bearing words such as adjectives and verbs in the same dependency structure. For example, in "*The food was delicious*", the adjective "delicious" is linked to the aspect "food." Similarly, in "*I loved the restaurant*", the verb "loved" is linked to "restaurant." The system then calculates a sentiment score using VADER's lexicon-based polarity scores, adjusted by contextual modifiers (intensifiers like "very" or diminishers like "slightly") and negations (e.g., "not good").

The sentiment of each aspect is further refined by averaging conjunct sentiments (e.g., “cozy and welcoming”) and aggregating all scores associated with the same aspect. Finally, the system classifies each aspect’s overall sentiment into negative, neutral, or positive categories based on the aggregate score.

This approach has the advantage of being interpretable and lightweight, as it doesn’t require training or large datasets. However, its performance is limited by lexical coverage and dependency accuracy; it may struggle with figurative language, domain-specific terms, or implicit sentiments.

2.2 TransformerABSA

The TransformerABSA model uses advanced deep learning to analyze opinions in text. It identifies what people are talking about (the aspects) and determines whether their opinion about each aspect is positive, negative, or neutral. The model works in two main steps: aspect extraction and sentiment classification.

In the first step, the system finds aspect terms in the sentence using a pre-trained transformer called `gauneg/roberta-base-absa-ate-sentiment`. This model labels each word with a tag that shows whether it belongs to an aspect or not. The implementation uses the Hugging Face *token-classification* pipeline, which automatically groups together related tokens to form complete aspect phrases such as “food” or “customer service.”

In the second step, the model determines how people feel about each aspect. It uses another pre-trained transformer, `yangheng/deberta-v3-base-absa-v1.1`, which performs sentiment classification. For every aspect found in the text, the model reads both the full sentence and the aspect name together, for example: `[CLS] The food was amazing but the service was slow. [SEP] service [SEP]`. Based on this input, it predicts whether the sentiment about “service” is positive, neutral, or negative.

Finally, the model combines the results from both steps and produces structured output such as `[("food", "positive"), ("service", "negative")]`. This approach allows TransformerABSA to understand complex language, context, and even sarcasm better than simple rule-based systems. However, because it depends on large pre-trained models, it is slower and requires more computational power than methods like LexiconABSA.

2.3 LLMABSA

The LLMABSA approach uses Large Language Models (LLMs), which are very powerful AI models trained on vast amounts of text data, to automatically understand and analyze customer reviews.

Instead of manually defining rules or training specific models for aspect extraction and sentiment classification, the LLM reads the entire review in context, recognizes important features (or aspects) like “food,” “service,” or “atmosphere,” and determines the sentiment whether it’s positive, negative, or neutral expressed towards each.

This method uses the deep understanding that LLMs have about language, including sarcasm and nuanced expressions, which helps in catching complex or implicit sentiments that other methods might miss.

The LLM also requires less set-up since it doesn't need specialized training for this task. However, it may require powerful hardware, and sometimes its answers need fine-tuning or additional processing to align perfectly with specific domain needs.

In summary, LLMABSA leverages the state-of-the-art capabilities of language models to deliver a flexible and context-aware sentiment analysis solution that adapts well across diverse kinds of review data.

Design Decisions

The project was structured into modular components to keep the code clean and easy to maintain. Each ABSA model (LexiconABSA, TransformerABSA, and LLMABSA) was implemented as a separate class following the same interface, which made comparison consistent. In the Transformer implementation, we used a separate model for aspect extraction and another for sentiment classification to improve accuracy and flexibility. Helper functions were added for data preprocessing, metric calculation, and visualization, while caching and error handling were used to manage long LLM runs and parsing issues. This modular and reusable setup made evaluation faster, clearer, and easier to extend.

Experimental Setup

Data

We use real-world user reviews collected from relevant domains such as product feedback and customer service platforms (in our case it's restaurant reviews). The data contains varied language styles and covers multiple aspects typically mentioned by users. This diversity ensures that the methods are tested on realistic and challenging input.

To measure and compare the performance of the different ABSA methods, we use standard metrics:

Precision and Recall: To assess how accurately each method detects relevant aspect terms and sentiment polarities.

F1 Score: The harmonic mean of precision and recall, providing a balanced metric for overall effectiveness.

Accuracy: Percentage of correctly classified sentiment labels.

Resource Usage: Comparison of computational costs such as inference time, model size, and memory consumption to evaluate efficiency.

Test Scenarios

We tested our three ABSA methods on real user reviews to see how well they find aspects (like "food" or "service") and understand the sentiment (positive, negative, or neutral) expressed about each.

Our tests check if the methods correctly identify aspects, label their sentiment properly, and handle tricky cases like multiple opinions in one sentence, slang, or negations. We also tested the structure and format of the output to make sure it's consistent.

We used real samples and evaluation data and ran many tests to compare accuracy and reliability. This helps us understand which method works best for different kinds of reviews and situations.

Results & Analysis:

Qualitative comparison

The lexicon approach performs well on simple sentences and is transparent in how it derives sentiment, but struggles with complex language and sarcasm.

The transformer model handles context better and picks up nuanced sentiments more accurately.

The LLM approach excels in understanding subtle, implicit sentiments and complex expressions that other models miss, showing a deeper comprehension of language.

LexiconABSA Example:

"The food was amazing but the service was slow."

Our lexicon-based ABSA method extracted two aspects:

- Food, with a positive sentiment and moderate confidence.

- Service, with a neutral sentiment and very low confidence.

This shows the lexicon method can clearly recognize explicit positive terms linked to "food" (e.g., "amazing") and assign a positive feeling. However, for "service," no strong sentiment word was detected, so it is labeled neutral with low confidence.

This example illustrates the strength of the lexicon method in handling clear, direct sentiments, and its limitation when sentiment expressions are weak, ambiguous, or implicit because of narrow vocabulary.

TransformerABSA Example:

Text: "The battery life is terrible but the display is beautiful."

The model found two aspects:

- Battery life, labeled negative sentiment.

- Display, labeled positive sentiment.

This shows how the transformer model accurately detects multi-word aspects and their sentiments, understanding context better than simpler methods.

LLMABSA Example:

Text: "The battery life is terrible but the display is beautiful."

The model detected two aspects:

Battery life, which it marked as negative with a high confidence of 0.96.

Display, which it identified as positive with a confidence of 0.94.

This shows how the LLM can identify different aspects and their sentiments very accurately, along with how sure it is about each.

Quantitative comparison

This comparison looks at how well and how fast the three ABSA models work. The main metrics are precision, recall, F1-score, accuracy, and inference time (how long each model takes to make a prediction).

Model	Precision	Recall	F1-score	Accuracy	Avg Confidence	Avg Inference Time (s)
Lexicon	1.00	0.510	0.675	0.510	NaN	0.144
Transformer	1.00	0.706	0.828	0.706	NaN	0.342
LLM	1.00	0.804	0.891	0.804	0.327	10.644

Speed:

The Lexicon model is the fastest, taking only about 0.13 seconds per review. It's good for quick or large-scale analysis.

The Transformer model is a bit slower (≈ 0.3 seconds per review) but still fast enough for most uses.

The LLM model is much slower (≈ 12 seconds per review) because it's a very large model that processes text in a more complex way.

Accuracy:

The Lexicon model has the lowest accuracy ($F1 = 0.66$) because it only looks for known words and doesn't understand context.

The Transformer model performs much better ($F1 = 0.81$) since it captures the meaning of words in context.

The LLM model gives the best results ($F1 = 0.89$), correctly identifying both the aspects and their sentiments most of the time.

Resources:

The Lexicon model uses very little memory and runs on any machine.

The Transformer needs more memory but still runs easily on a normal computer.

The LLM model uses a lot of memory and is much heavier to run.

Conclusion:

There is a clear trade-off:

The **Lexicon model** is fast but not very accurate.

The **Transformer model** gives a good balance between speed and quality.

The **LLM model** is the most accurate but also the slowest and most resource-intensive.

Discussion

Strengths and weaknesses of each approach

Lexicon-based model:

This method is the fastest and easiest to run, using very little computing power. It works well for quick or large-scale sentiment analysis. However, it only relies on predefined word lists and simple rules, so it often misses sarcasm, mixed opinions, or context. As a result, it performs poorly when the language becomes more complex or indirect.

Transformer-based model:

The transformer understands context much better and can recognize how words change meaning in different sentences. It gives more accurate results than the lexicon model and keeps a good balance between speed and performance. Still, it needs more resources and might require extra training or fine-tuning to reach its full potential.

LLM-based model:

The LLM produces the most accurate and detailed results. It can understand deep meaning, sarcasm, and subtle language cues better than the other two. However, it takes much longer to run and needs much more memory. This makes it better suited for smaller-scale or research-oriented tasks where accuracy matters more than speed.

Use case recommendations

Lexicon-based model:

Best for fast, lightweight sentiment analysis where results don't need to be very detailed. It's suitable for real-time monitoring, such as tracking quick trends on social media, product reviews, or customer feedback dashboards. Because it runs easily on low resources, it's ideal for simple applications or embedded systems.

Transformer-based model:

A good choice for balanced performance and accuracy. It works well for customer review analysis, business intelligence, or feedback categorization where context matters but speed is still important. It's suitable for deployment in production systems that can handle moderate computational loads.

LLM-based model:

Best for high-accuracy, in-depth sentiment understanding where nuance and reasoning matter, for example, detailed brand analysis, research on opinion mining, or complex customer experience studies. It's especially useful for small-scale or offline use where processing time is less critical, and accuracy is the main goal.

Challenges faced and solutions

Lexicon-based model:

Our main issue was that it sometimes missed context or gave wrong sentiment when the wording was sarcastic or indirect. It also produced more neutral predictions than needed.

Solution: We improved text preprocessing and normalization (like removing determiners and fixing token splits), which helped the model detect aspects more reliably. To solve it completely, you can combine it with a lightweight contextual model and expand the sentiment lexicon with domain-specific words.

Transformer-based model:

The transformer performed well but was sensitive to sentence structure and sometimes missed combined aspects (like "battery life" or "camera quality"). Also, we also struggled to find a good solution to extract sentiments because the model didn't do it itself.

Solution: We optimized input formatting and added helper functions and another model to extract aspects more consistently. If you want to achieve a perfect sentiment detection, fine-tune the model on a domain-specific dataset, use smaller optimized models,

LLM-based model:

The LLM gave the most accurate results but often took a long time to respond and sometimes failed to return proper JSON output. In some runs, it even stopped due to low memory.

Solution: We added error handling in the code to catch parsing issues and limited the number of test samples to avoid memory crashes. To save some time, use a lighter model or apply model compression (distillation or quantization), set stricter output validation rules, and cache responses to speed up repeated runs.

Conclusion:

Key learnings and potential improvements

The main thing we learned from this project is that each approach has its own pros and cons, so no one solution works best for every case. The lexicon-based model is really fast and uses little memory, making it great if you need quick results and don't mind a bit less accuracy. But it's not good with tricky or complex expressions because it just looks up words in a dictionary.

The Transformer-based model gave a good balance between speed and accuracy. It understood sentence context much better and handled most opinions correctly, though it sometimes missed multi-word aspects or neutral tones. It would likely improve further with more domain-specific data or fine-tuning on a larger dataset.

The large language model gave the most accurate results because it understands language deeply, but it was much slower and used a lot of memory. This makes it hard to use in settings where you need fast results.

In the future, a hybrid approach could combine the lexicon's speed with the transformer or LLM's contextual understanding. The transformer could also be improved using data augmentation, fine-tuning, or smaller distilled versions of large models to improve efficiency without losing much accuracy.

So, choosing a method depends on what we need speed, accuracy, or a mix of both. There's no perfect choice yet, but each method has its use cases and potential to get better.